

Group Project: Scattergories

Assigned: 2026-06-24 · **Due:** 2026-06-25 · **Team size:** 3

Purpose

- To practice **PostgreSQL** and **Prisma** by building a **multi-user game**.
- To practice **table relationships**, **constraints**, and validating untrusted **user input** on the server.
- To work as a **team** on one codebase, splitting the backend, the data, and the client.
- To have fun.

The Game

Based on the party game **Scattergories**: each game has a **letter** and a **topic** (category). Players race to submit an answer in that topic that starts with the letter. First to claim an answer keeps it; duplicates are rejected.

- Players join with a **room code** and a **username** (no accounts, no auth)
- The server is the referee: it decides what counts as a valid, unique answer

Task

Build an **Express API** (Prisma + PostgreSQL) with at least these routes:

- **New game:** `POST /games` with a `roomCode`; the server randomly picks a **letter** and a **topic**, creates the game, and returns them
- **List games:** show all games with their **room code**, **letter**, and **topic**
- **Submit answer:** a player sends `{ roomCode, username, answer }`

Rules the server enforces

- The **room code** must be **unique**: if a game with that `roomCode` already exists, reject it (e.g. `409`)
- The answer must **start with the game's letter**, or respond with an error (e.g. `400`)
- The answer must be **new for that room**: if it was already submitted, reject it (e.g. `409`)
- **No auth** (just to keep things simple): the **username** is only a label, and we rely on players to use their own name honestly

Relationship (required)

- A **game** has many **answers** (`Game` 1:N `Answer`)
- Enforce “no duplicate answer in a room” with a **UNIQUE** constraint on `(game_id, answer)`, then have Express catch that error and reject the submission
- Make `room_code` **UNIQUE** too, so two games can't share a code

Exposing Your Server

So classmates can play together, expose your Express API with **ngrok**:

```
ngrok http 3000
```

ngrok creates a temporary **public URL** that tunnels requests to a port on your machine (here, your API on `localhost:3000`), so people on the internet can reach a server running on your laptop.

Share that URL; players send their answers there. Expose the **API**, never your database directly.

Example Exchanges

```
POST /games { "roomCode": "PLUM42" }
{ "roomCode": "PLUM42", "letter": "B", "topic": "Animals" }
```

```
POST /games { "roomCode": "PLUM42" }
409 { "error": "Room code already in use" }
```

```
GET /games
[
  { "roomCode": "PLUM42", "letter": "B", "topic": "Animals" },
  { "roomCode": "KIWI88", "letter": "S", "topic": "Foods" }
]
```

```
POST /answers { "roomCode": "PLUM42", "username": "sam", "answer": "Bear" }
{ "accepted": true }
```

```
POST /answers { "roomCode": "PLUM42", "username": "kai", "answer": "Bear" }
409 { "error": "Answer already taken" }
```

```
POST /answers { "roomCode": "PLUM42", "username": "kai", "answer": "Cat" }
400 { "error": "Answer must start with B" }
```

Sharing the Database with pgAdmin

Your teammates need the same tables and starter data. Use pgAdmin to move a database between machines:

- **Export:** right-click your database, choose **Backup...**, and save a dump file (schema plus data)
- **Import:** create an empty database, right-click it, choose **Restore...**, and select that dump file
- Commit the dump into your repo's `data/` folder so everyone restores the same starting point

Official guide: https://www.pgadmin.org/docs/pgadmin4/latest/backup_and_restore.html

Repo Structure

Keep the project organized, for example:

```
scattergories/
├─ api/          # Express + Prisma server (the game API)
├─ client/       # React app (tiered goal)
├─ data/         # database dump to import (schema + starter data)
└─ README.md    # how to import the data and run everything
```

Your `README` should clearly explain:

- How to **import the database** (restore the dump in `data/` with pgAdmin)
- How to set up `.env` (your `DATABASE_URL`)
- How to **run the API** (and the client, if you build one), and how to expose it with ngrok

Requirements and Allowances

- Tracked in a **GitHub repo from the start**, with **regular commits from all members**.
- Use **Prisma + PostgreSQL**; the `Game 1:N Answer` relationship is required.
- You may use **AI** at any stage, but **disclose** how and where you used it.
- You may use **third-party packages**; note which ones and why.

Deliverables

- A link to your **GitHub repository**.
- A working **Express + Prisma API** that runs the game and enforces the rules.
- A short **README**: how to run it, how to expose it with ngrok, and your AI-use disclosure.

Submission

- Push your final work to GitHub.
- Submit the repo link via the **repo submission form** in slack.

Tiered Learning Goals

- **Normalize input**: store answers in a consistent form (lowercase, trimmed), so “Bear” and “ bear ” count as the same.
- **Add a timer**: store a **timestamp** when the game starts, and reject answers submitted more than **5 minutes** later.
- **Scoring**: award more points for **longer answers**, and return a per-player score.
- **Persistent scoreboard**: add a `scores` table (a new table in Postgres) that tracks totals across games.
- **Validate real words**: check each answer against the free **Dictionary API** (no key): `GET https://api.dictionaryapi.dev/api/v2/entries/en/<word>` returns `200` if the word exists, `404` if not
- **A React client**: a web UI to create games and submit answers (reuse your College Schedule Builder skills).
- **Deploy** the API so it is online without ngrok.
- **Auth**: add real player accounts and logins.